COMPUTER ENGINEERING

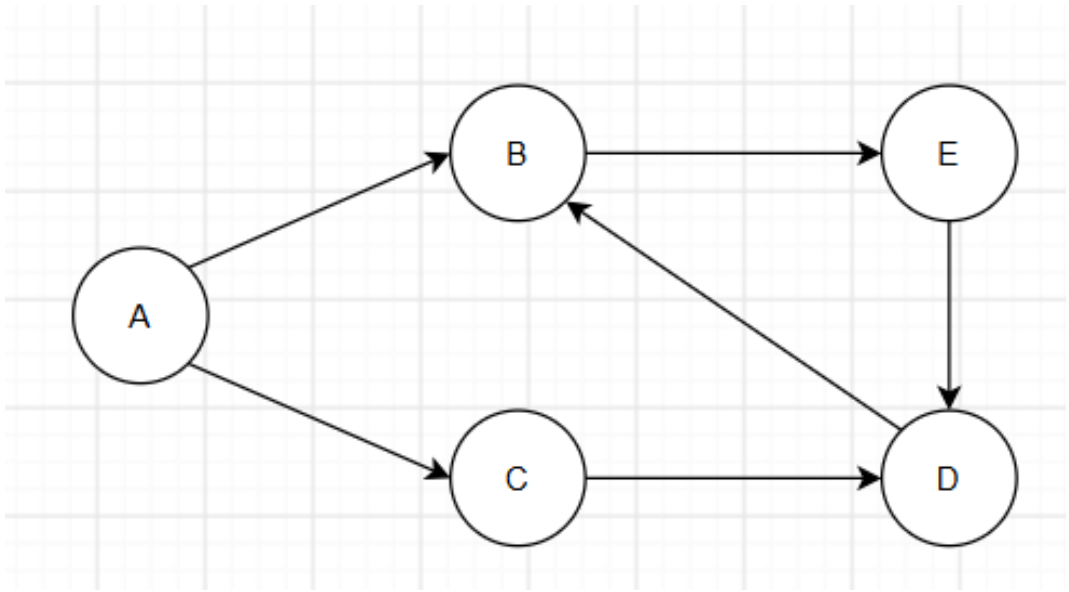
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/02/2026	**DATE SUBMITTED:**	10/02/2026
NAME:	Moradillo, Eunice E.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA	
GRAPH	

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

CODE

```cpp
#include <iostream>
using namespace std;

// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;

        return newNode;
    }

    int N;

public:
    adjNode** head;

    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;

        head = new adjNode*[N];

        for (int i = 0; i < N; ++i) {
            head[i] = nullptr;
        }

        for (int i = 0; i < n; ++i) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);

            head[start_ver] = newNode;
        }
    }
```

```cpp
    ~DiaGraph() {
        for (int i = 0; i < N; ++i) {
            adjNode* current = head[i];

            while (current != nullptr) {
                adjNode* next = current->next;
                delete current;
                current = next;
            }
        }

        delete[] head;
    }
};

// A.1
void displayVertices(int N) {
    cout << "Vertices: ";

    for (int i = 0; i < N; ++i) {
        cout << char('A' + i);

        if (i < N - 1)
            cout << ", ";
    }

    cout << endl;
}

// A.2
void displayAdjMatrix(graphEdge edges[], int n, int N) {
    int matrix[10][10] = {};

    for (int i = 0; i < n; ++i) {
        int start = edges[i].start_ver;
        int end = edges[i].end_ver;
        int weight = edges[i].weight;

        matrix[start][end] = weight;
    }
```

```cpp
    cout << "\nAdjacency Matrix\n\n";

    cout << "  ";
    for (int i = 0; i < N; ++i) {
        cout << char('A' + i) << " ";
    }
    cout << endl;

    for (int i = 0; i < N; ++i) {
        cout << char('A' + i) << " ";

        for (int j = 0; j < N; ++j) {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

// A.3
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        cout << "(" << char('A' + i) << ", "
             << char('A' + ptr->val) << ", "
             << ptr->cost << ")" << endl;

        ptr = ptr->next;
    }
}

int main() {

    // Graph edges
    graphEdge edges[] = {
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6;

    int n = sizeof(edges) / sizeof(edges[0]);
```

```cpp
    // A.1 Create a Graph
    cout << "A.1 Create a Graph" << endl;
    displayVertices(N);

    // A.2 Adjacency Matrix
    cout << "\nA.2 Adjacency Matrix" << endl;
    displayAdjMatrix(edges, n, N);

    // A.3 Adjacency List
    cout << "\nA.3 Adjacency List" << endl;
    cout << "(Start Vertex, End Vertex, Weight)" << endl;

    for (int i = 0; i < N; ++i) {
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
```

OUTPUT

```
A.1 Create a Graph
Vertices: A, B, C, D, E, F

A.2 Adjacency Matrix

Adjacency Matrix

  A B C D E F
A 0 2 4 0 0 0
B 0 0 0 0 3 0
C 0 0 0 2 0 0
D 0 4 0 0 0 0
E 0 0 0 3 0 0
F 0 0 0 0 0 0

A.3 Adjacency List
(Start Vertex, End Vertex, Weight)
(A, C, 4)
(A, B, 2)
(B, E, 3)
(C, D, 2)
(D, B, 4)
(E, D, 3)
```

OBSERVATION:

The graph is directed and weighted, so the adjacency matrix isn't symmetric. A-B has weight 2, but B-A is 0. Vertex F has no edges, so its row and column are all zeros. In the adjacency list, the edges of A print as (A, C, 4) before (A, B, 2). This is because each new node is inserted at the head of the linked list, so the edges come out in reverse order of insertion.

ILOB	
CODE	

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;  // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);  // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

OUTPUT

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\euinie\Documents\CPE010\FINALS\HOA10\ILOB\output>
```

OBSERVATION:
Starting from vertex 1, the DFS visit order was 1, 5, 8, 6, 7, 3, 4, 2. The stack (LIFO) makes the most recently pushed neighbor get explored first. Vertex 5 was pushed after vertex 2, so it was visited first, and the search kept going deeper along one path before backtracking. The visited set prevents a vertex from being processed twice, even if it was pushed more than once.

ILOB2	
CODE	

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;  // FIX: was missing, causes undefined behavior
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```cpp
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // FIX: renamed to U, reusing T here shadows the class template parameter (compile error)
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1);  // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

OUTPUT

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\euinie\Documents\CPE010\FINALS\HOA10\ILOB2\output>
```

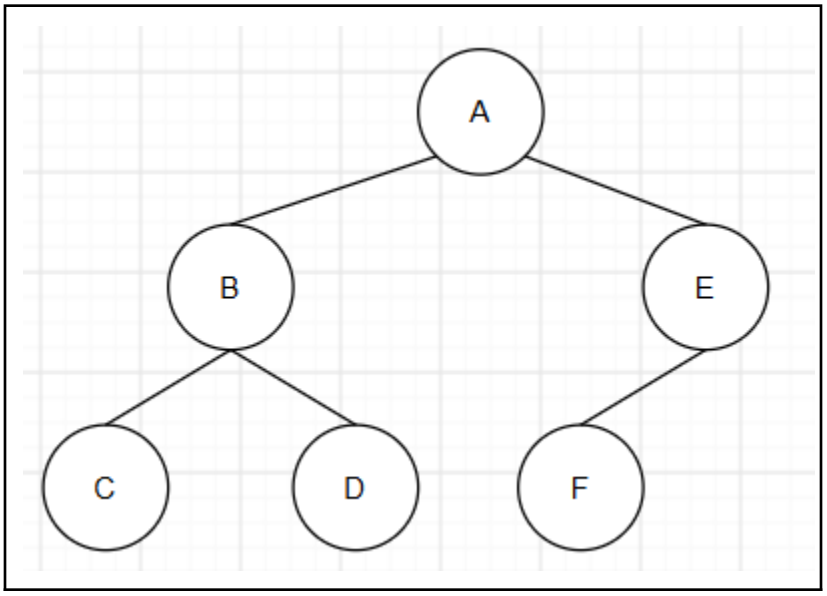
OBSERVATION:
Starting from node1, the BFS visit order was 1, 2, 5, 4, 8, 3, 6, 7. The queue (FIFO) makes the traversal go level by level. First come the neighbors of 1 (2 and 5), then their neighbors (4 and 8), and so on. Compared to DFS on the same graph structure, BFS spreads out wide instead of going deep. The edge weights are stored in the graph, but neither traversal uses them.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task**
   - The algorithm that would be the most helpful to accomplish this task is the Depth First Search. It is because it does backtracks unlike Breath First Search,DFS follows one path until there are no unvisited neighbors, backtracks along the same path, and picks the next unexplored branch from the same vertex.
2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**



Tree preorder: A B D E C F	**Graph DFS:** A B D E C F
Tree preorder(node): if node is null: return visit(node)	**DFS** dfs(v): mark v as visited for each unvisited u

for each child c of node: preorder(c)	to v: dfs(u)

```cpp
#include <iostream>
#include <map>
#include <set>
#include <string>

std::map<char, std::string> g = {{'A',"BC"}, {'B',"ADE"}, {'C',"AF"}, {'D',"B"}, {'E',"B"}, {'F',"C"}};
std::set<char> seen;

void dfs(char v) {
    seen.insert(v);                          // mark v as visited
    std::cout << v << " ";
    for (char u : g[v]) if (!seen.count(u)) dfs(u);
}

struct Node { char val; Node *l, *r; };

void preorder(Node* n) {
    if (!n) return;                          // empty, stop
    std::cout << n->val << " ";              // visit node
    preorder(n->l); preorder(n->r);          // go to each child
}

int main() {
    Node F{'F'}, D{'D'}, E{'E'}, C{'C',&F}, B{'B',&D,&E}, A{'A',&B,&C};
    std::cout << "Graph DFS: ";     dfs('A');      std::cout << "\n";
    std::cout << "Tree Preorder: "; preorder(&A);  std::cout << "\n";
}
```

```
Graph DFS: A B D E C F
Tree Preorder: A B D E C F
```

3. **In the performed code, what data structure is used to implement the Breadth First Search?**
   - In the performed code in the procedure, I use a queue as a data structure that follows the FIFO rule in the ILOB2 to implement the Breadth First Search.
4. **How many times can a node be visited in the BFS?**
   - A node in Breath-Fist Search or BFS can only be visited once because of level by level order where each node will be dequeued after a node is visited then it will mark as visited, then it will explore all the adjacent nodes then the remaining nodes in the queue. It can only visit once because, as I said, it marks after visiting to avoid revisiting and infinite looping.

CONCLUSION

In this activity, I implemented a graph using an adjacency matrix and an adjacency list, then used the same graph structure to perform DFS and BFS. I learned that the data structure used decides how the traversal behaves. DFS uses a stack and goes deep before backtracking, while BFS uses a queue and visits vertices level by level. I also saw that both need a visited set to avoid revisiting vertices and looping forever. The tree preorder and graph DFS gave the same order (A B D E C F), which shows that preorder traversal is the tree equivalent of DFS.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.